

Lo que el temario de *Web client and backend development frameworks* da por visto y nunca vuelve a explicar. El nivel base, antes de la Unidad I.

Busca un término... ej. DOM, POO, llave foránea, MVC

Todo

● Cómo funciona la web

● El navegador

● POO y Java

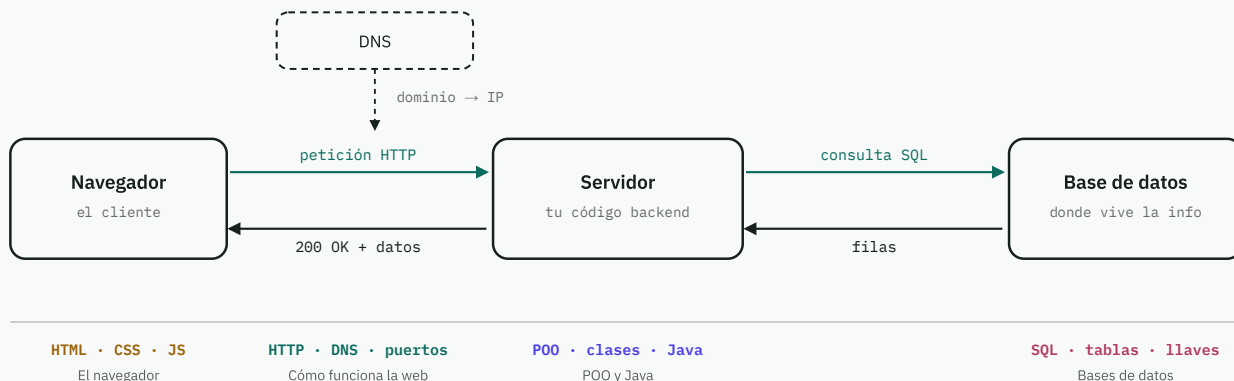
● Bases de datos

● Herramientas

El programa oficial dice que esta materia va **después** de *Tecnologías para el desarrollo de aplicaciones web* y *Software quality assurance and design patterns*. Todo lo de esta página viene de esas dos: el profesor asume que ya lo traes y construye encima sin volver a tocarlo.

No son 84 cosas por aprender de cero. Son 84 palabras que, cuando las oigas en clase, no deberían frenarte. Si entiendes el diagrama de aquí abajo y las cinco secciones marcadas **crítico**, ya vas a poder seguir el hilo de la Unidad I.

Qué pasa cuando escribes una dirección y das Enter



Cada tramo del recorrido corresponde a una sección de esta página.

Este recorrido es el mapa mental que la materia da por hecho. Casi todo el semestre consiste en cambiar una pieza de este dibujo: partir el servidor en varios (microservicios), meterlo en una caja (contenedores), o cambiar cómo el navegador pide los datos (API REST).

BASE 1 CRÍTICO

Cómo funciona la web

Sin esto, la **Unidad II (APIs)** y la **Unidad III (backend)** no tienen dónde apoyarse. Es la sección que más rinde si solo tienes un rato.

Cliente

El programa que pide algo: normalmente el navegador o una app de celular.

Piénsalo como: el comensal que hace el pedido.

Servidor

La computadora (y el programa corriendo en ella) que recibe peticiones y responde. No es una caja mágica: es tu código esperando llamadas.

Piénsalo como: la cocina que recibe la comanda y devuelve el platillo.

Petición y respuesta request / response

El único patrón de comunicación de la web: el cliente pide, el servidor contesta. Siempre empieza el cliente; el servidor nunca habla primero.

Piénsalo como: la cocina jamás te manda comida que no pediste.

HTTP

El idioma con reglas fijas en que cliente y servidor se hablan. Define cómo se escribe una petición y cómo se escribe una respuesta.

Piénsalo como: el formato obligatorio de la comanda.

HTTPS HTTP + TLS

HTTP con el contenido cifrado en el camino, para que nadie en medio pueda leerlo ni modificarlo.

Piénsalo como: mandar la comanda en un sobre sellado en vez de gritarla.

URL

La dirección completa de un recurso. Tiene partes: `https://` (protocolo) + `sitio.com` (dominio) + `/productos/5` (ruta) + `?orden=precio` (query string).

Piénsalo como: país + calle + número + instrucciones para el repartidor.

Dominio

El nombre legible del sitio (`google.com`). Existe solo porque a las personas se les olvidan los números.

DNS

El servicio que traduce un dominio a la dirección IP real del servidor. Pasa antes de que salga la primera petición.

Piénsalo como: la agenda de contactos que convierte un nombre en un número de teléfono.

Dirección IP

El número que identifica de forma única a una máquina en la red.

Puerto

Un número que distingue qué programa dentro de esa máquina atiende la conexión. La web usa 80 (HTTP) y 443 (HTTPS) por default.

Piénsalo como: la IP es el edificio, el puerto es el número de departamento.

localhost 127.0.0.1

Tu propia máquina, vista desde ella misma. Cuando corres el servidor en tu laptop, ahí es donde lo vas a encontrar.

Piénsalo como: mandarte una carta a tu propia casa.

Headers cabeceras

Información extra que acompaña a la petición o respuesta sin ser el contenido: quién eres, qué formato aceptas, tu token.

Piénsalo como: los datos del sobre — remitente, destinatario, "frágil" — no la carta.

Body cuerpo

El contenido real que viaja en la petición o respuesta: el JSON, el HTML, el archivo.

Piénsalo como: la carta dentro del sobre.

Cookie

Un dato chiquito que el servidor le pide al navegador guardar, y que el navegador reenvía solo en cada petición al mismo sitio.

Piénsalo como: el sello de mano en un antro — te lo ponen al entrar y lo enseñas cada vez.

Código de estado 200, 404, 500

Número de 3 dígitos con el que el servidor resume qué pasó. `2xx` salió bien, `4xx` te equivocaste tú, `5xx` se equivocó el servidor.

Piénsalo como: 404 es "ese platillo no existe en el menú"; 500 es "se incendió la cocina".

Frontend

Todo lo que corre en el dispositivo del usuario y se ve en pantalla.

Backend

Todo lo que corre en el servidor: lógica, reglas, acceso a datos. El usuario nunca lo ve directamente.

API Interfaz de Programación

Un conjunto de funciones que un programa expone para que otro programa las use. En web, casi siempre significa "endpoints que devuelven JSON".

Piénsalo como: el menú — la lista de lo que puedes pedir, sin ver cómo se cocina.

BASE 2 CRÍTICO

El navegador: HTML, CSS y JavaScript

Toda la **Unidad IV (frontend)** asume esto. React y Angular son capas encima de estas tres cosas — si no las traes, los frameworks se sienten mágicos e incontrolables.

HTML

El lenguaje de etiquetas que define la *estructura* y el contenido de una página: títulos, párrafos, botones, formularios.

Piénsalo como: los huesos y órganos.

CSS

El lenguaje que define cómo se *ve* ese HTML: colores, tamaños, posiciones, espaciado.

Piénsalo como: la ropa y el peinado.

JavaScript

El lenguaje de programación que corre dentro del navegador y hace que la página *reaccione*: responde clics, pide datos, cambia el contenido sin recargar.

Piénsalo como: los músculos y el sistema nervioso.

Etiqueta / elemento tag

Cada pieza de HTML, escrita entre `<` y `>`.
Casi siempre abre y cierra: `<p>texto</p>`.

Atributo

Información extra dentro de una etiqueta: `class`, `id`, `href`. Sirve para identificarla o configurarla.

DOM Document Object Model

El árbol de objetos que el navegador construye a partir del HTML. JavaScript no modifica el archivo HTML: modifica este árbol, y la pantalla se actualiza sola.

Piénsalo como: el HTML es el plano en papel; el DOM es la maqueta armada que sí puedes mover.

Selector CSS

La forma de apuntar a qué elementos aplicar un estilo: por etiqueta (`p`), por clase (`.boton`) o por id (`#menu`).

Evento

Algo que pasa en la página y a lo que tu código puede reaccionar: un clic, escribir en un campo, cargar la página.

Piénsalo como: un timbre; tú decides qué hacer cuando suena.

DevTools F12 · inspeccionar

Las herramientas del navegador para ver el DOM, los estilos, los errores de JS y las peticiones de red. Es donde vas a depurar todo el semestre.

Piénsalo como: la radiografía de la página.

Consola

La pestaña de DevTools donde aparecen los errores y donde `console.log()` imprime lo que le mandes.

Asíncrono

Código que no bloquea: pide algo que tarda (una petición de red) y sigue trabajando; cuando llega la respuesta, la atiende. Esencial en web porque la red siempre tarda.

Piénsalo como: dejar la ropa en la lavandería y seguir tu día, en vez de quedarte parado viéndola girar.

Promesa Promise

El objeto de JavaScript que representa "un resultado que todavía no llega, pero llegará (o fallará)".

Piénsalo como: el ticket de la lavandería.

async / await

Sintaxis que deja escribir código asíncrono como si fuera línea por línea, sin anidar callbacks. Es la forma moderna de esperar una promesa.

fetch

La función de JavaScript para hacer una petición HTTP desde el navegador. Es literalmente cómo el frontend habla con tu API.

Piénsalo como: el teléfono con el que la mesa llama a la cocina.

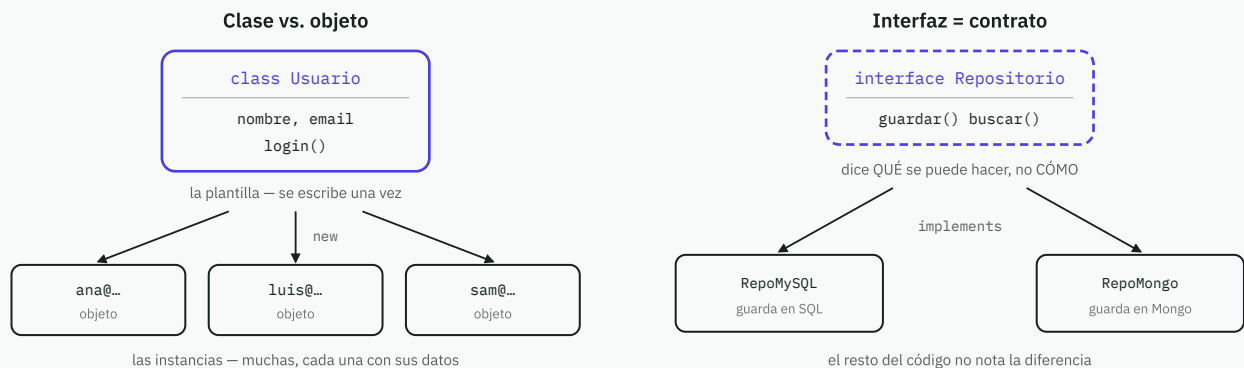
AJAX

Nombre viejo de la técnica de pedir datos al servidor sin recargar la página. Hoy se hace con `fetch`, pero el profe puede decirle AJAX.

BASE 3 CRÍTICO

Programación orientada a objetos y Java

La bibliografía trae Spring e Hibernate. La **Unidad III (ORM)** es, literalmente, mapear clases a tablas — si "clase" y "objeto" no son firmes, el ORM no puede tener sentido.



La interfaz es la idea que hace posibles la arquitectura hexagonal y la de cebolla de la Unidad I: el dominio depende del contrato, no de la base de datos concreta.

Clase

La plantilla que describe qué datos y qué comportamiento tendrá algo. Se escribe una vez.

Piénsalo como: el molde de galletas.

Objeto instancia

Una copia concreta creada a partir de una clase, con sus propios valores.

Piénsalo como: cada galleta salida del molde.

Atributo campo, propiedad

Un dato que guarda el objeto: `nombre`,
`email`.

Método

Una función que pertenece a la clase y actúa sobre sus datos: `usuario.login()`.

Constructor

El método especial que se ejecuta al crear el objeto, para dejarlo listo con sus valores iniciales.

Encapsulamiento

Esconder los datos internos y obligar a que se accedan por métodos, para que nadie los deje en un estado inválido.

Piénsalo como: el cajero del banco — no te dejan meter la mano en la bóveda.

Herencia

Una clase toma los atributos y métodos de otra y les agrega o cambia cosas.

Piénsalo como: `Admin` es un `Usuario` con permisos extra; no reescribes al usuario.

Interfaz

Un contrato: lista de métodos que una clase promete tener, sin decir cómo los implementa. La base de casi todos los patrones de diseño.

Piénsalo como: el enchufe de pared — define la forma, no quién genera la luz.

Polimorfismo

Poder tratar objetos distintos de la misma forma porque cumplen el mismo contrato, y que cada uno responda a su manera.

Piénsalo como: le dices "habla" a un perro y a un gato; ambos obedecen, cada quien con su sonido.

Paquete package · namespace

Carpeta lógica que agrupa clases relacionadas y evita choques de nombres.

Excepción try / catch

La forma de manejar errores sin que el programa muera: intentas algo, y si falla, capturas la falla y decides qué hacer.

Piénsalo como: la red de seguridad del trapecista.

Anotación @Entity, @GetMapping

Etiqueta que pones sobre una clase o método en Java para que el framework le dé un trato especial. Spring e Hibernate se manejan casi todo así.

Piénsalo como: un post-it que dice "esto es una tabla" y el framework lo lee y actúa.

Tipado estático

El lenguaje exige declarar el tipo de cada dato y revisa los errores al compilar, antes de correr. Java y TypeScript son así; JavaScript no.

Piénsalo como: revisar el equipaje antes de salir, no al llegar al aeropuerto.

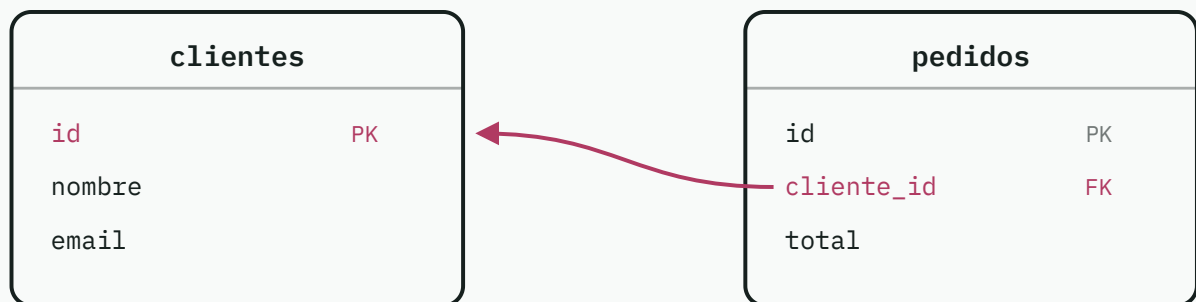
JDK / JVM

El JDK son las herramientas para escribir y compilar Java; la JVM es la máquina virtual que ejecuta el resultado en cualquier sistema operativo.

BASE 4 CRÍTICO

Bases de datos y SQL

La **Unidad III** arranca directo en "mapeo objeto-relacional". Ese nombre solo tiene sentido si ya sabes qué es lo relacional que se está mapeando.



cliente_id apunta a clientes.id

un cliente tiene muchos pedidos

esa relación es la que el ORM traduce a objetos

Los datos no se repiten: el pedido no guarda el nombre del cliente, guarda una referencia a su fila. Reconstruir la información completa es lo que hace un `JOIN` — y lo que un ORM automatiza.

Base de datos relacional

Almacén donde la información vive en tablas con columnas fijas, y las tablas se conectan entre sí por referencias.

Piénsalo como: un archivero con folders de formato idéntico y referencias cruzadas.

Tabla, fila, columna

La tabla es el conjunto (ej. `clientes`); la columna es un campo (`email`); la fila es un registro concreto (un cliente).

Esquema schema

La definición de qué tablas y columnas existen y de qué tipo son. En relacional se define antes de guardar nada.

Llave primaria PK

La columna que identifica de forma única cada fila. No se repite y no puede estar vacía.

Piénsalo como: la CURP de la fila.

Llave foránea FK

Una columna que apunta a la llave primaria de otra tabla. Es el mecanismo con el que se relacionan.

Piénsalo como: en tu acta de nacimiento viene la CURP de tu mamá, no toda su biografía.

SQL

El lenguaje para hablarle a una base de datos relacional: pedir, insertar, actualizar y borrar datos.

Consulta query

Una instrucción SQL. La más común: `SELECT * FROM clientes WHERE id = 5`.

CRUD

Las cuatro operaciones básicas sobre datos: Crear, Leer (Read), Actualizar y Borrar (Delete). Los verbos HTTP de la Unidad II mapean casi uno a uno con esto.

Piénsalo como: POST=Crear, GET=Leer, PUT=Actualizar, DELETE=Borrar.

JOIN

Operación SQL que combina filas de dos tablas siguiendo su relación, para armar la información completa.

Piénsalo como: juntar el folder del pedido con el folder del cliente para tener la historia entera.

Transacción

Un grupo de operaciones que se aplican todas o ninguna. Si algo falla a medias, todo se revierte.

Piénsalo como: una transferencia — no puede salir el dinero de una cuenta sin entrar a la otra.

Índice

Estructura extra que hace que buscar por cierta columna sea rápido, a costa de algo de espacio.

Piénsalo como: el índice al final del libro — evita leer todas las páginas.

NoSQL no relacional

Bases de datos que no usan tablas fijas. La más común en este curso es MongoDB, que guarda documentos tipo JSON con estructura libre.

Piénsalo como: una caja de notas donde cada nota puede tener campos distintos.

BASE 5 CRÍTICO

Las herramientas del día a día

El perfil docente del programa exige control de versiones y control de dependencias. Las 12 prácticas se entregan con esto, y la **Unidad V** lo lleva a contenedores y despliegue.

Terminal línea de comandos, CLI

La ventana donde escribes comandos en texto en vez de dar clics. Casi toda herramienta seria de desarrollo se maneja desde aquí.

Git

Sistema de control de versiones: guarda el historial de tu código, te deja volver atrás y trabajar en paralelo sin pisarte con otros.

Piénsalo como: puntos de guardado en un videojuego, pero con ramas alternas.

Repositorio repo

El proyecto completo con todo su historial de Git.

Commit

Un punto guardado en el historial, con un mensaje que explica qué cambió.

Branch rama

Una línea de trabajo paralela, para desarrollar algo sin tocar la versión estable.

Piénsalo como: una copia del borrador para experimentar sin arruinar el original.

Merge

Unir los cambios de una rama con otra. Si dos personas tocaron lo mismo, hay que resolver el conflicto a mano.

GitHub

Servicio en la nube donde se hospedan repositorios de Git para compartirlos y colaborar. Git es la herramienta; GitHub es el sitio.

Dependencia

Código de terceros que tu proyecto necesita para funcionar. Nadie escribe todo desde cero.

Gestor de dependencias

`npm` · `Maven` · `Gradle`

Herramienta que descarga e instala las dependencias en las versiones correctas.

`npm` en el mundo JavaScript; `Maven` o `Gradle` en Java.

Piénsalo como: el que va al súper por los ingredientes de la receta.

`package.json` / `pom.xml`

El archivo donde se declara qué dependencias necesita el proyecto. El primero es de Node; el segundo, de Maven/Java.

Piénsalo como: la lista de ingredientes de la receta.

Librería vs. framework

Una librería la llamas tú cuando la necesitas. Un framework te llama a ti: él manda la estructura y tú rellenas los huecos.

Piénsalo como: la librería es una herramienta que tomas; el framework es la casa ya construida donde acomodas tus muebles.

IDE VS Code, IntelliJ

Editor de código con extras: autocompletado, detección de errores, depurador integrado.

Variable de entorno

Un valor de configuración que vive fuera del código (contraseñas, URLs de la BD), para no subirlo al repositorio ni recompilar al cambiarlo.

Piénsalo como: las llaves no van pegadas a la puerta.

Build compilación

El paso que convierte tu código fuente en algo ejecutable o listo para el navegador.

Runtime

El entorno donde el código ya compilado se ejecuta. Node.js es el runtime de JavaScript fuera del navegador; la JVM lo es de Java.

BASE 6

Patrones de diseño

Vienen de la materia antecedente de *design patterns*. Toda la **Unidad I** son patrones aplicados a escala de arquitectura, así que estos nombres van a caer de entrada. Es la sección menos urgente: se puede ir ganando sobre la marcha.

Patrón de diseño

Una solución ya probada a un problema que se repite en el diseño de software, con nombre propio para poder discutirla rápido.

Piénsalo como: una jugada ensayada — no la inventas en la cancha, y todos saben cómo se llama.

Acoplamiento

Qué tanto depende una pieza de código de otra. Mucho acoplamiento significa que cambiar una cosa rompe cinco más.

Piénsalo como: focos conectados en serie — se funde uno y se apagan todos.

Cohesión

Qué tan enfocada está una pieza en una sola responsabilidad. Alta cohesión es buena: la clase hace una cosa y la hace bien.

MVC Modelo-Vista-Controlador

Separa los datos (modelo), la presentación (vista) y quien coordina las peticiones (controlador). Es el patrón base de casi todo framework web.

Piénsalo como: el modelo es la bodega, la vista es el menú, el controlador es el mesero.

DAO / Repositorio

Una clase cuya única tarea es leer y escribir en la base de datos, para que el resto del código no sepa SQL.

Piénsalo como: el bodeguero — le pides lo que necesitas y no te enteras de cómo está acomodada la bodega.

DTO objeto de transferencia

Un objeto simple que solo carga datos de un lado a otro, sin lógica. Se usa para decidir qué campos expone una API y cuáles no.

Piénsalo como: el sobre en el que mandas solo los papeles necesarios, no el expediente entero.

Inyección de dependencias DI

En vez de que una clase cree sus propias dependencias, se las pasan desde afuera. Es el corazón de Spring y lo que hace testeable la arquitectura hexagonal.

Piénsalo como: al cocinero le entregan los ingredientes; no lo mandas al súper cada vez.

Singleton

Patrón que garantiza que de cierta clase exista una sola instancia en todo el programa.

Piénsalo como: solo hay un director de la escuela.

Factory fábrica

Una clase que se encarga de crear objetos, para que quien los pide no tenga que saber qué tipo concreto se construye.

Piénsalo como: pides "un café" y la barra decide qué máquina usar.

SOLID

Cinco principios de diseño orientado a objetos. El que más va a salir: *una clase, una responsabilidad, y depende de interfaces, no de implementaciones.*